Jaypee Univeristy of Information Technology, Waknaghat Solan H.P

Solution Sheet
Fibonacci Heap

Q1) Answer: a
Removing the minimum root node and consolidating the remaining trees in a Fibonacci heap takes O(log n) amortized time because, after removing the minimum root, its children are added to the root list, and trees of the same degree are repeatedly merged until each root has a unique degree. The key observation is that a node's degree is at most O(log n), since Fibonacci heap trees grow according to Fibonacci-number bounds. Using the potential method, the amortized cost of consolidation is O(log n), even if an individual operation performs more work. Thus, extracting the minimum element takes O(log n) amortized time.

Q2) Answer: b
To union two Fibonacci heaps, you simply concatenate their root lists and update the global minimum pointer in (O(1) worst-case time.

Q3) Answer: c
In a Fibonacci heap, each node keeps a mark to remember whether it has already lost one child.

- When a node loses its first child, it is marked.
- If it loses a second child, it is cut from its parent and moved to the root list.

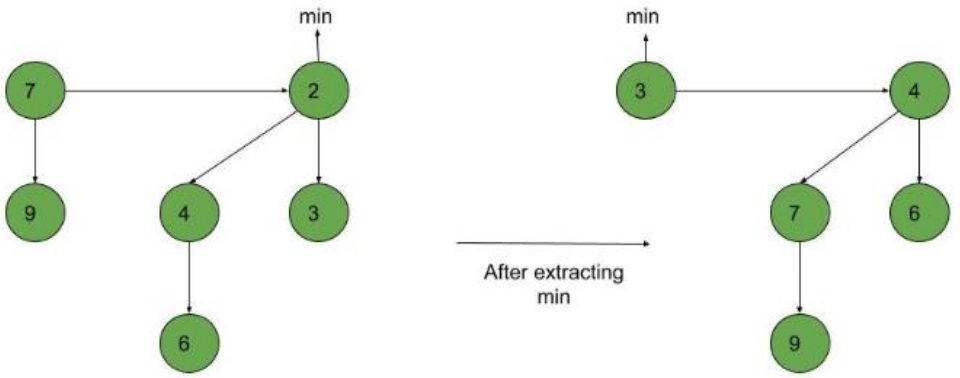
This prevents trees from becoming too weak or unbalanced. The marks also help us prove that the heap's operations have good, amortized time complexity.

Q4) Answer: b
All n nodes can be roots after individual insertions as we insert only in root linked list. After Extract-Min, consolidation links trees of equal degree, leaving O(log n) roots.

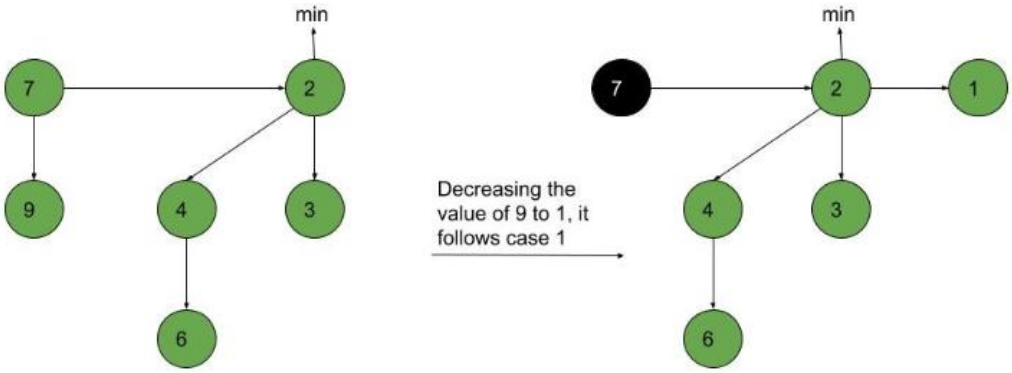
Q5) Answer: d
The main characteristics of a Fibonacci heap is violated since min[H] must contain one with smallest value.

Q6) For a Fibonacci heap insertion, union take O(1) while remaining take O(log n) time.

Q7)

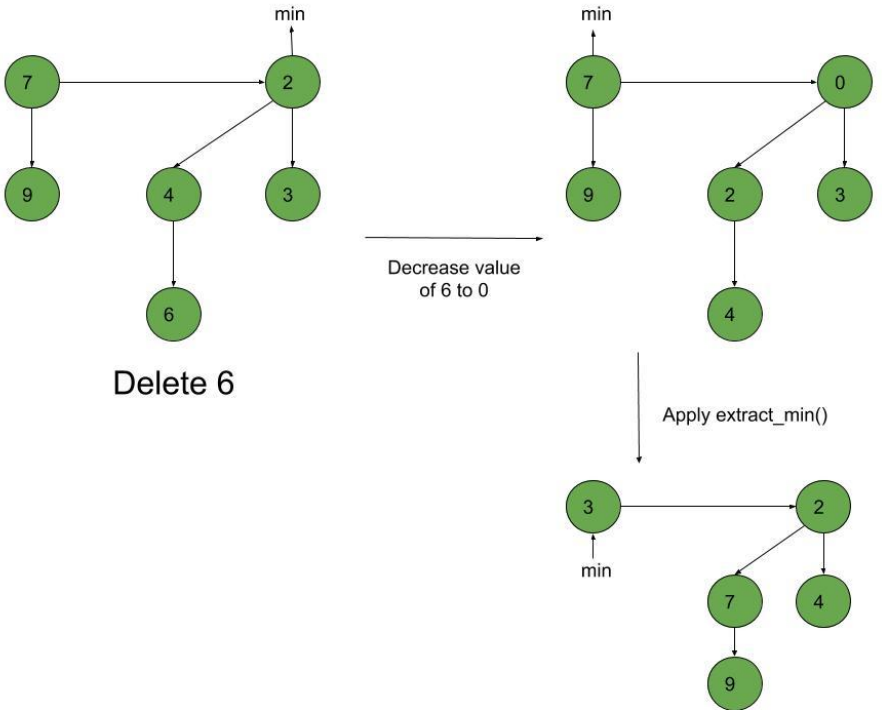


Q8)

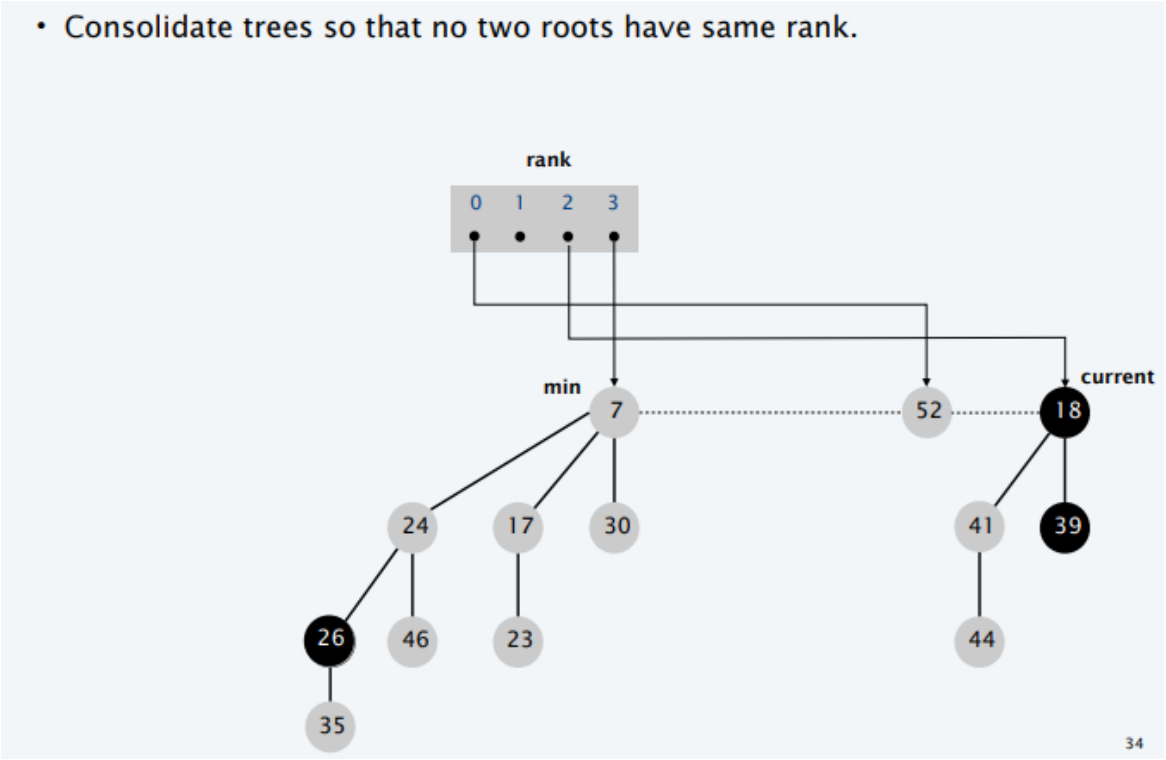


After above steps we have to update the min pointer as 1 is the smallest value in the given heap.

Q9)



Q10) Final heap after consolidation step.



Reference: Kaplan, H. and Tarjan, R.E., 1999. *New heap data structures*. Technical Report TR-597-99, Department of Computer Science, Princeton University.